

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4

Тестування та контроль якості (QA) вбудованих систем

«Служби DNS та NAT. Підходи до траблшутінгу мережі»

Виконав
студент групи ІА–34:
Оборський Д.В.

Перевірила:
Міщенко Л. Д.

Київ 2026

Мета роботи: Ознайомитися з технологією NAT. Налаштувати NAT за допомогою iptables. Розібратися з підходами до траблшутингу мережі.

Посилання на репозиторій: https://github.com/EcchiGrill/qa-lab-4

Теоретична підготовка

NAT (Network Address Translation) - механізм заміни IP-адрес і портів у заголовках пакетів під час їх проходження через маршрутизатор. Основне призначення NAT - подолання нестачі адрес IPv4: вузли локальної мережі використовують приватні адреси (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), які не маршрутизуються в Інтернеті, а назовні всі вони представлені однією зовнішньою адресою маршрутизатора. Вузли в Інтернеті бачать лише адресу маршрутизатора, тому внутрішня структура мережі прихована.

Розрізняють кілька видів трансляції. SNAT (Source NAT) замінює адресу джерела вихідних пакетів на задану статичну адресу. MASQUERADE - різновид SNAT, при якому адресою джерела стає поточна адреса вихідного інтерфейсу; він зручний, коли зовнішня адреса отримується динамічно (через DHCP або Wi-Fi). DNAT (Destination NAT) замінює адресу призначення і використовується для переадресації портів (port forwarding). Якщо кілька внутрішніх вузлів одночасно використовують одну зовнішню адресу, маршрутизатор додатково підміняє порти (для ICMP - ідентифікатор запиту), щоб розрізняти сеанси - це PAT (NAPT, NAT overload).

Netfilter - підсистема ядра Linux, що обробляє пакети, а iptables - утиліта командного рядка для керування її правилами (потребує прав root). Правила згруповано в таблиці (filter, nat, mangle, raw, security), а таблиці - у ланцюжки. У таблиці nat використовуються ланцюжки PREROUTING (DNAT - до прийняття рішення про маршрутизацію), POSTROUTING (SNAT / MASQUERADE - перед відправленням пакета з інтерфейсу) та OUTPUT (для пакетів самого хоста). Без вказання ключа -t команди працюють з таблицею filter. У сучасному Arch Linux команда iptables за замовчуванням працює через бекенд nf_tables (iptables-nft), але синтаксис правил не змінюється.

Трансляцію адрес виконує модуль відстеження з'єднань conntrack: для першого пакета сеансу правило nat створює запис трансляції, а всі наступні пакети цього сеансу, зокрема відповіді, перетворюються автоматично за цим записом (зворотна заміна адреси призначення). Таблицю трансляцій можна переглянути командою conntrack -L. Щоб Linux-хост пересилав пакети між інтерфейсами, необхідно увімкнути маршрутизацію в ядрі параметром net.ipv4.ip_forward = 1; інакше пакети, адресовані не самому хосту, відкидаються.

Network namespace - механізм ядра Linux, що створює ізольований мережевий стек зі своїми інтерфейсами, адресами, таблицею маршрутизації та правилами Netfilter. Пара віртуальних інтерфейсів veth працює як Ethernet-кабель: кадр, надісланий в один кінець, з'являється на іншому. Розмістивши один кінець пари в окремому namespace, на одному комп'ютері можна отримати повноцінний другий вузол мережі.

Траблшутинг NAT зручно виконувати знизу вгору за рівнями моделі OSI, локалізуючи проблему на кожному кроці: локальний зв'язок між клієнтом і NAT-маршрутизатором (ARP, ping), доступ маршрутизатора до Інтернету, таблиці маршрутизації та шлюзи за замовчуванням, коректність параметрів DHCP і DNS, стан служб, правила iptables і параметр ip_forward, і нарешті доступність DNS-сервера з клієнта. Аналізуючи захоплений трафік у Wireshark на внутрішньому та зовнішньому інтерфейсах, можна точно визначити, на якому етапі пакет зникає або не транслюється.

Конфігурація середовища

Роботу виконано на одному ноутбуці з операційною системою Arch Linux. Методичка передбачає два комп'ютери: PC1 підключений до Інтернету та до PC2 через Ethernet, а PC2 - лише до PC1. Оскільки другого комп'ютера немає, PC2 змодельовано за допомогою мережевого простору імен (network namespace) pc2, з'єднаного з основною системою парою віртуальних інтерфейсів veth (віртуальний Ethernet-кабель). Основна система ноутбука виконує роль PC1: NAT-маршрутизатора та DHCP-сервера для внутрішньої мережі. Відповідно до номера варіанту внутрішня мережа - 10.0.15.0/24. Зовнішній інтерфейс PC1 - wlan0 у домашній Wi-Fi-мережі 192.168.50.0/24.

Параметр	**PC1 (ноутбук, основна система)**	**PC2 (namespace pc2)**
Роль	NAT-маршрутизатор, DHCP-сервер	Клієнт внутрішньої мережі
ПЗ	Arch Linux, iptables (nf_tables), ISC DHCP (dhcpd), conntrack-tools, Wireshark	Той самий Arch Linux, ізольований мережевий стек; dhclient
Інтерфейси	wlan0 - зовнішній (Інтернет) veth-pc1 - внутрішній	veth-pc2
IPv4	wlan0: 192.168.50.32/24 veth-pc1: 10.0.15.1/24 (статично)	10.0.15.10 - 10.0.15.20 (через DHCP від PC1)

Параметр	PC1 (ноутбук, основна система)	PC2 (namespace pc2)
Шлюз за замовчуванням	192.168.50.1 (домашній роутер)	10.0.15.1 (PC1)
DNS	192.168.50.1	192.168.50.1, 8.8.8.8 (через DHCP)

Хід роботи

1. Підготовка: перевірка ПЗ та параметрів мережі PC1

Перевіряю версію iptables і бекенд, встановлюю conntrack-tools та net-tools (для команди route), визначаю адресу інтерфейсу wlan0 та шлюз за замовчуванням.

```
iptables --V
sudo pacman -S --needed conntrack-tools net-tools
ip -4 addr show wlan0
ip route
```

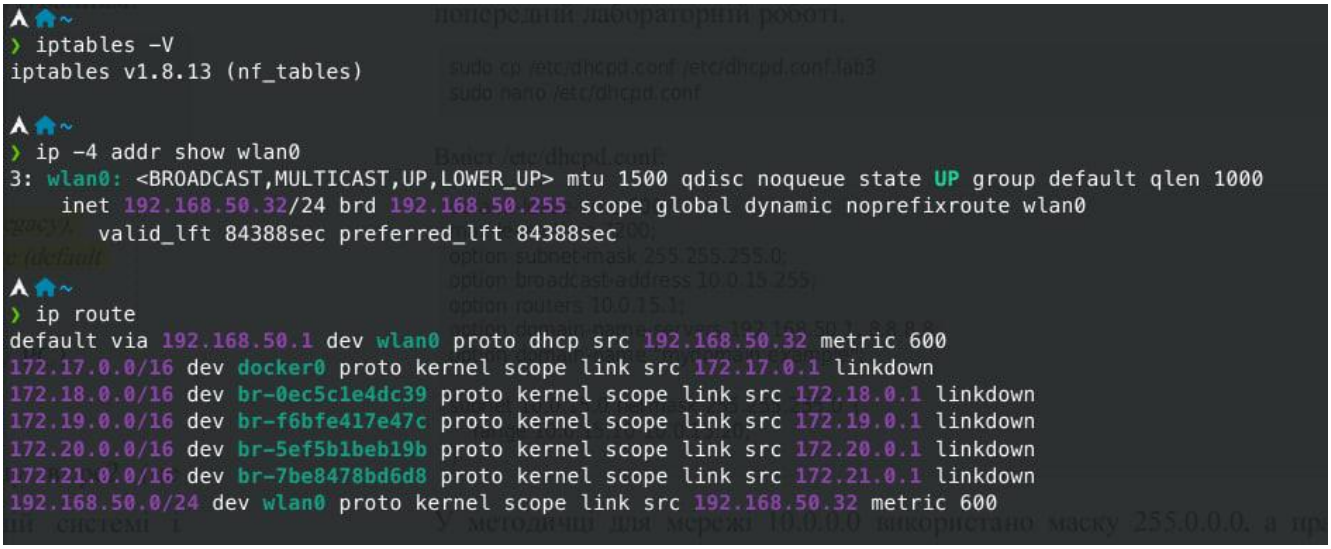


Рис. 1. Версія iptables та параметри зовнішнього інтерфейсу PC1

2. Створення PC2 у мережевому просторі імен

Створюю namespace pc2 і пару veth: кінець veth-pc2 переношу в pc2 (це «мережева карта» PC2), а кінець veth-pc1 залишаю в основній системі і призначаю йому адресу шлюзу внутрішньої мережі 10.0.15.1/24.

```
sudo ip netns add pc2
sudo ip link add veth-pc1 type veth peer name veth-pc2
sudo ip link set veth-pc2 netns pc2
sudo ip addr add 10.0.15.1/24 dev veth-pc1
sudo ip link set veth-pc1 up
sudo ip netns exec pc2 ip link set lo up
sudo ip netns exec pc2 ip link set veth-pc2 up
ip -br addr
sudo ip netns exec pc2 ip -br addr
```

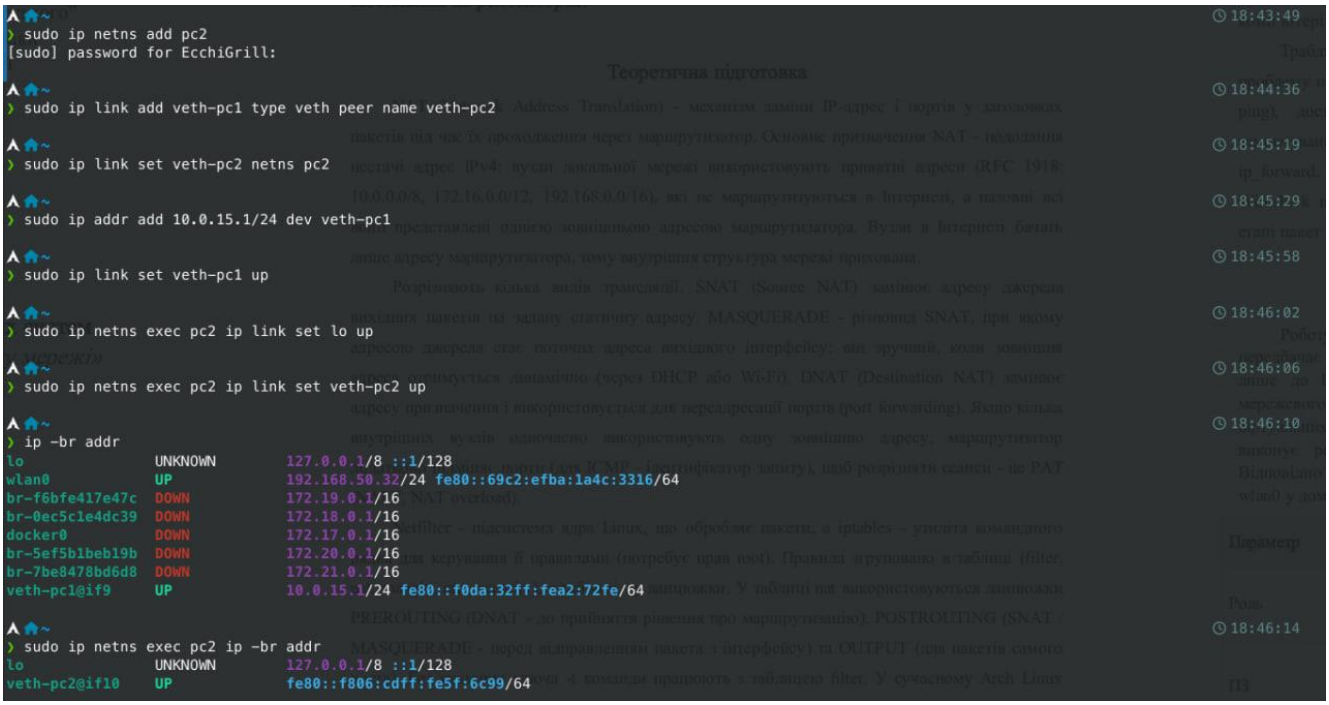


Рис. 2. Створення віртуального вузла PC2 та з’єднання veth

3. Налаштування DHCP-сервера для внутрішньої мережі

Використовую ISC dhcpd, але тепер він обслуговує лише внутрішню мережу 10.0.15.0/24. Оголошення subnet для 192.168.50.0/24 відсутнє, тому dhcpd не відповідає на запити в Wi-Fi-мережі і конфлікт двох DHCP-серверів з попередньої роботи не виникає. Шлюзом (option routers) для клієнтів є PC1 - 10.0.15.1. DNS-сервер 192.168.50.1 визначено у Wireshark.

```
sudo cp /etc/dhcpd.conf /etc/dhcpd.conf.lab4
sudo nano /etc/dhcpd.conf
```

Вміст /etc/dhcpd.conf:

```
default-lease-time 600;
max-lease-time 7200;
option subnet-mask 255.255.255.0;
option broadcast-address 10.0.15.255;
option routers 10.0.15.1;
option domain-name-servers 192.168.50.1, 8.8.8.8;
option domain-name "mydomain.example";

subnet 10.0.15.0 netmask 255.255.255.0 {
    range 10.0.15.10 10.0.15.20;
}
```

У методичці для мережі 10.0.0.0 використано маску 255.0.0.0, а правило NAT задано для 10.0.0.0/24. Щоб параметри DHCP і правило NAT описували одну й ту саму мережу, використовую маску /24 (255.255.255.0) для обох.

```
sudo dhcpd -t -cf /etc/dhcpd.conf
sudo systemctl restart dhcpd4
systemctl status dhcpd4 --no-pager
```

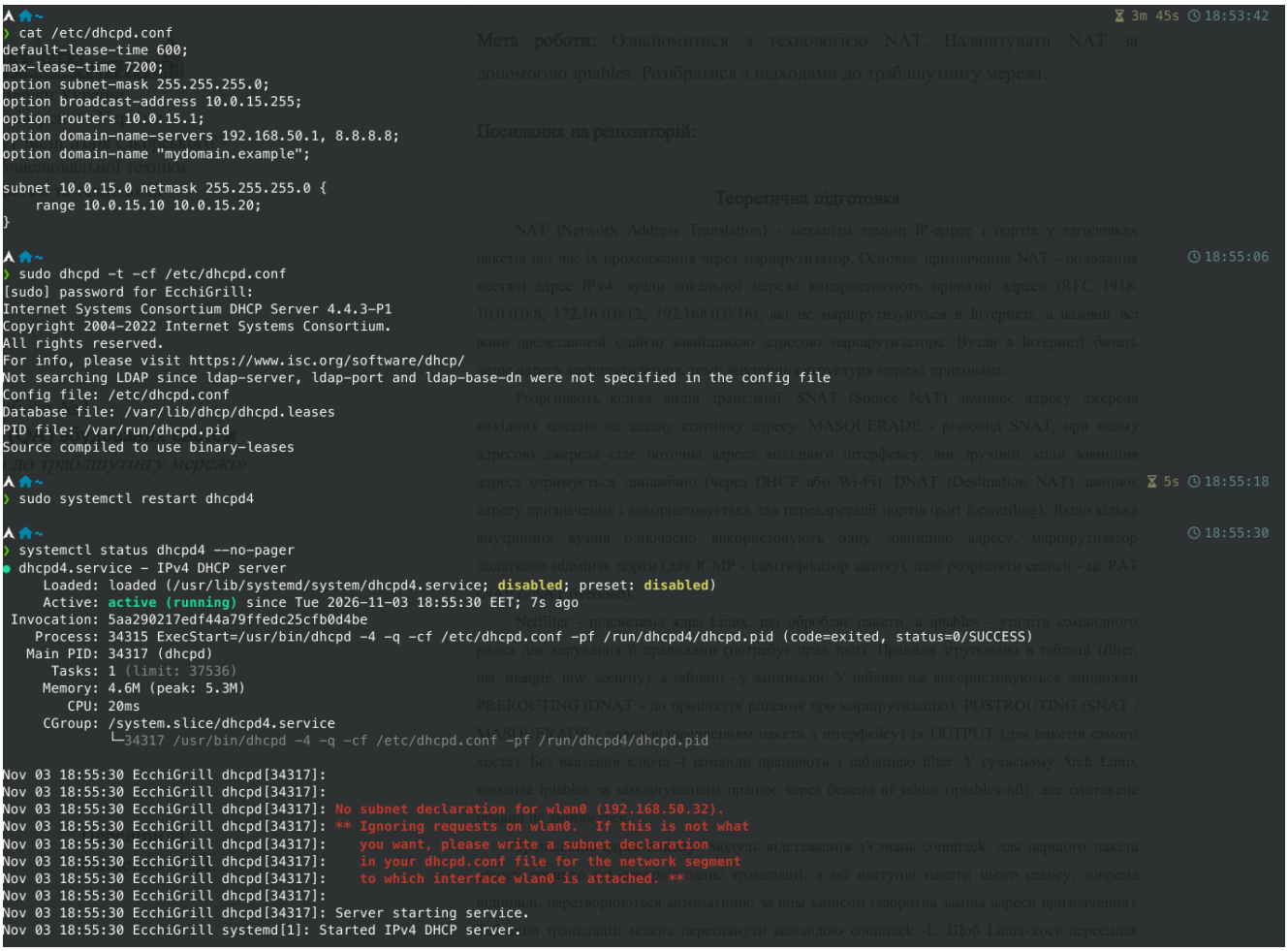


Рис. 3. Конфігурація та запуск DHCP-сервера на PC1

4. Отримання IP-адреси клієнтом PC2

Для pc2 створюю окремий файл /etc/netns/pc2/resolv.conf: команда ip netns exec підставляє його замість /etc/resolv.conf, тому dhclient у pc2 запише туди DNS-сервери, не змінюючи налаштувань основної системи. Для dhclient задаю окремі pid- та lease-файли, щоб він не конфліктував з клієнтом основної системи. Ця ж команда відповідає п. 8 траблшутингу (ручне отримання адреси клієнтом).

```
sudo mkdir -p /etc/netns/pc2
echo "nameserver 192.168.50.1" | sudo tee /etc/netns/pc2/resolv.conf
sudo ip netns exec pc2 dhclient -v -pf /run/dhclient-pc2.pid \
     -lf /var/lib/dhclient/dhclient-pc2.leases veth-pc2
sudo ip netns exec pc2 ip -4 addr show veth-pc2
sudo ip netns exec pc2 cat /etc/resolv.conf
```

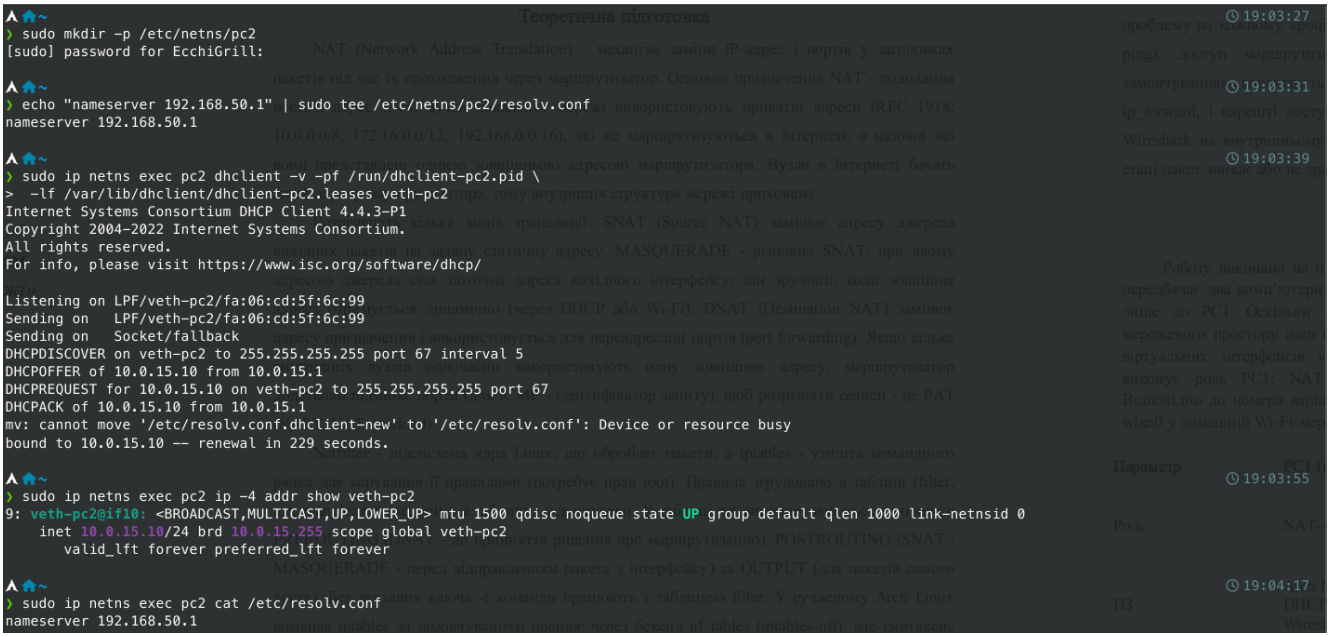


Рис. 4. Отримання IP-адреси клієнтом PC2 від DHCP-сервера PC1

5. Перегляд та очищення таблиці NAT

Переглядаю поточні правила таблиці nat. Ключ -n вимикає перетворення адрес в імена, -v показує лічильники пакетів. Якщо в таблиці є правила, очищаю її (sudo iptables -t nat -F). Увага: якщо на ноутбуці встановлено Docker або libvirt, очищення таблиці видалить і їхні правила.

```
sudo iptables -L
sudo iptables -t nat -L -n -v
sudo iptables -t nat -F
```



Рис. 5. Таблиця NAT до налаштування

Перевірка «до»: без NAT і маршрутизації PC2 не має доступу до Інтернету.

```
sudo ip netns exec pc2 ping -c 3 8.8.8.8
```

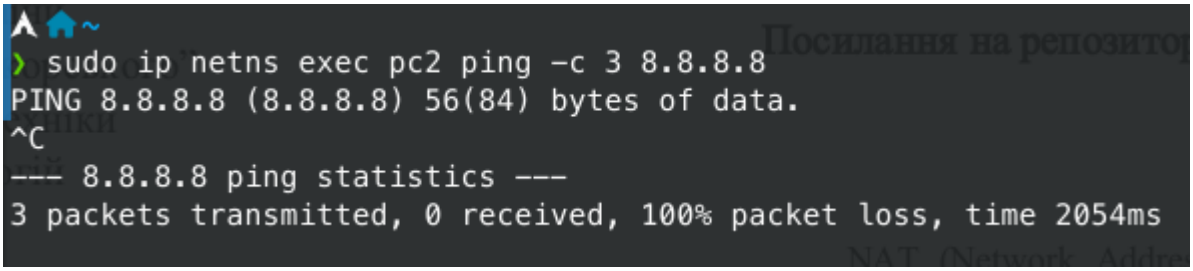


Рис. 6. Відсутність доступу до Інтернету з PC2 до налаштування NAT

6. Увімкнення маршрутизації в ядрі

Вмикаю пересилання пакетів між інтерфейсами та зберігаю параметр для наступних завантажень. В Arch Linux файл /etc/sysctl.conf не зчитується службою systemd-sysctl, тому параметр записую в окремий файл у каталозі /etc/sysctl.d/.

```
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/30-ipforward.conf
sudo sysctl --system | grep ip_forward
cat /proc/sys/net/ipv4/ip_forward
```

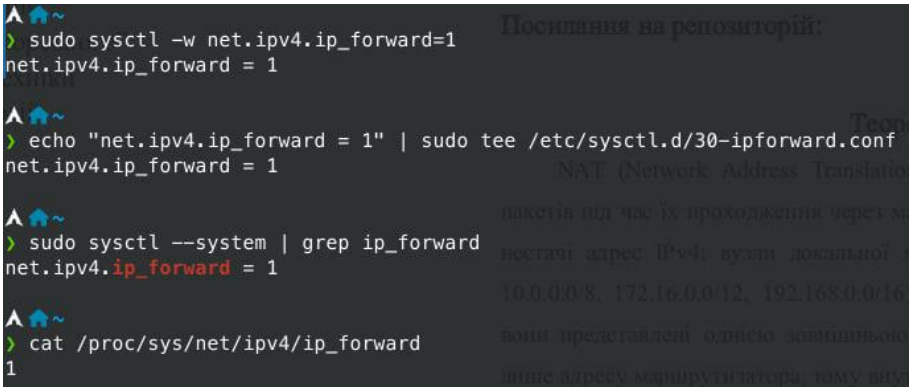


Рис. 7. Увімкнення IP-маршрутизації на PC1

7. Додавання правила NAT (MASQUERADE)

Додаю правило в ланцюжок POSTROUTING: пакети з внутрішньої мережі 10.0.15.0/24, що виходять через інтерфейс wlan0, отримують адресу джерела wlan0. Правило діє одразу після введення. Також перевіряю політику ланцюжка FORWARD таблиці filter: за замовчуванням в Arch вона ACCEPT, але якщо вона DROP (наприклад, через Docker), потрібно дозволити пересилання двома додатковими правилами.

```
sudo iptables -t nat -A POSTROUTING -o wlan0 -s 10.0.15.0/24 -j MASQUERADE
sudo iptables -L FORWARD -n -v
sudo iptables -A FORWARD -i veth-pc1 -o wlan0 -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o veth-pc1 -m conntrack --ctstate
RELATED,ESTABLISHED -j ACCEPT
sudo iptables -t nat -L -n -v
```

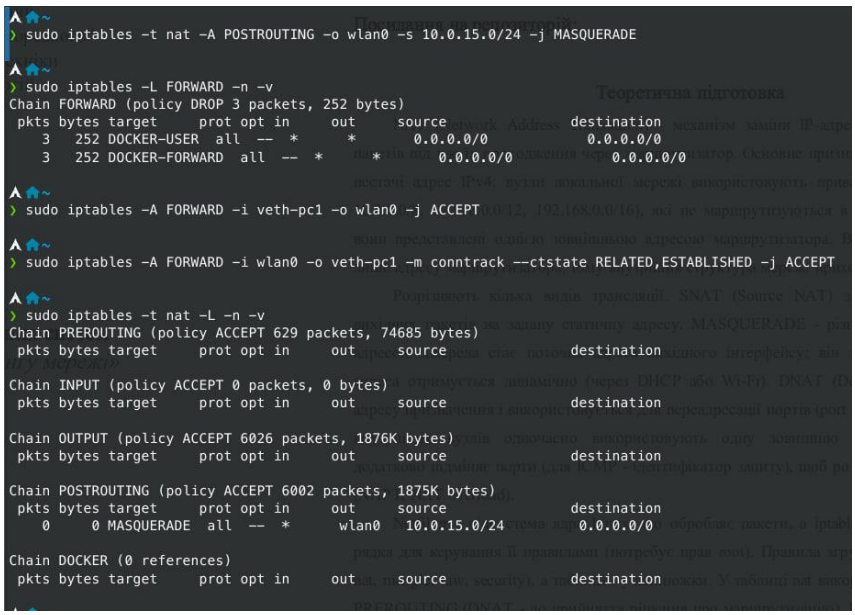


Рис. 8. Правило MASQUERADE у таблиці NAT

8. Збереження правил iptables

В Arch правила зберігаються у файл /etc/iptables/iptables.rules, який під час завантаження відновлює служба iptables.service.

```
sudo iptables-save -f /etc/iptables/iptables.rules
sudo cat /etc/iptables/iptables.rules
sudo systemctl enable iptables.service
```

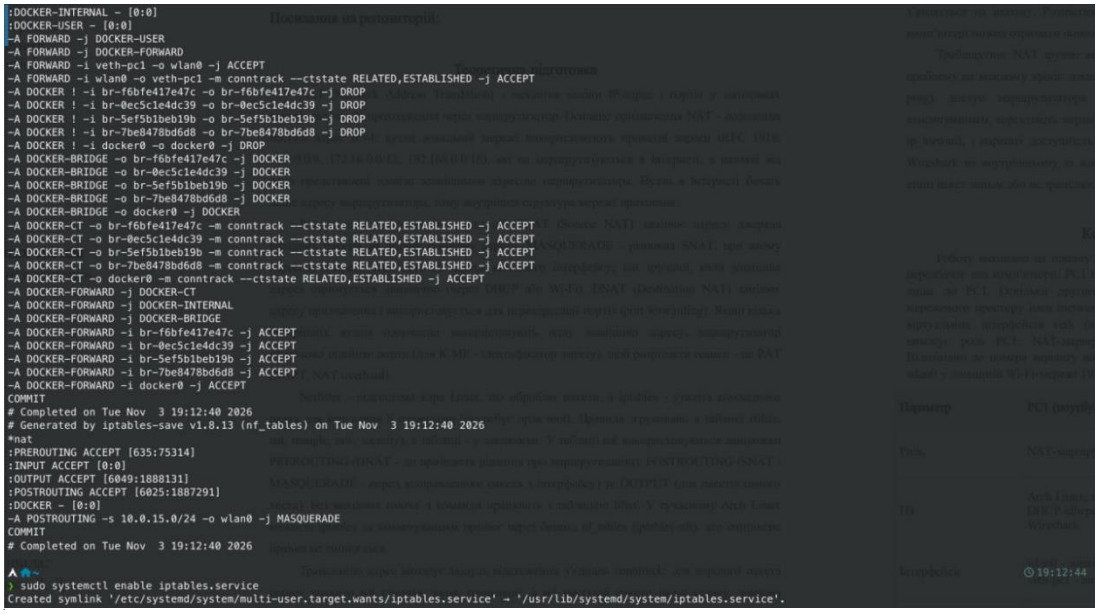


Рис. 9. Збереження правил iptables для автозавантаження

9. Перевірка роботи NAT

Запускаю Wireshark на PC1 і починаю захоплення одночасно на двох інтерфейсах - veth-pc1 (внутрішня мережа) та wlan0 (зовнішня), фільтр icmp. Додаю стовпець Interface (або запускаю два вікна Wireshark), щоб розрізняти інтерфейси. На PC2 виконую ping google.com.

```
sudo ip netns exec pc2 ping -c 5 google.com
```

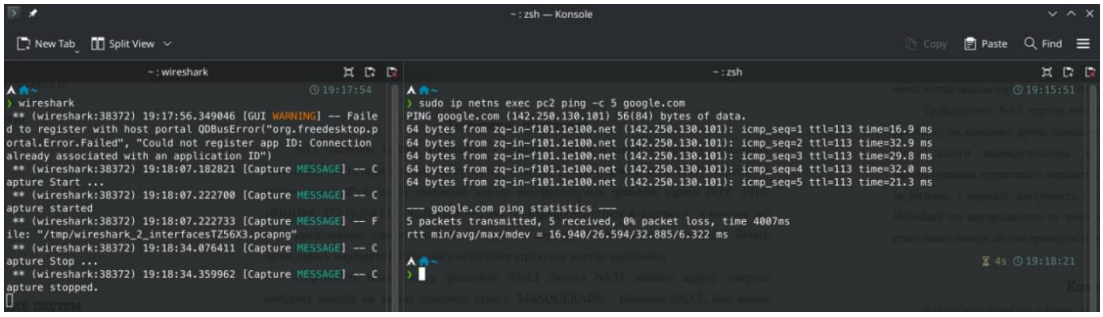


Рис. 10. Доступ до Інтернету з PC2 через NAT

На інтерфейсі veth-pc1 пакети мають адресу джерела PC2 (10.0.15.x), а на wlan0 ті самі ICMP-запити (з тими самими id та seq) мають адресу джерела 192.168.50.32 - адресу PC1. Відповіді на wlan0 приходять на 192.168.50.32, а на veth-pc1 вже адресовані 10.0.15.x. Це і є трансляція адрес.

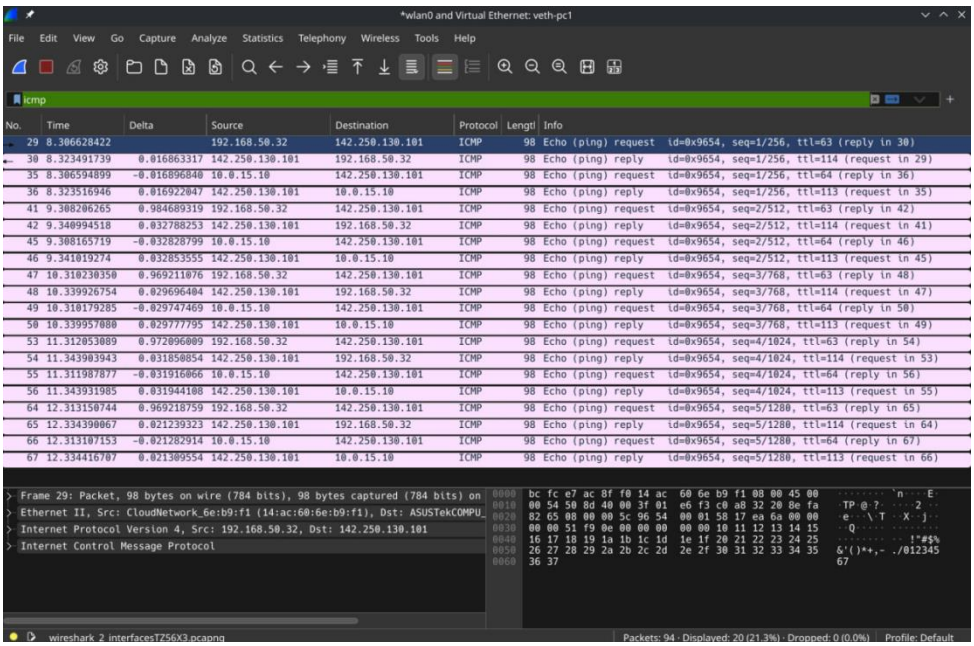


Рис. 11. ICMP-трафік PC2 на внутрішньому інтерфейсі veth-pc1та зовнішньому інтерфейсі wlan0 після трансляції

Таблицю трансляцій переглядаю під час виконання ping з PC2 (запускаю ping у другому терміналі):

```
sudo ip netns exec pc2 ping 8.8.8.8
sudo conntrack -L -p icmp
```

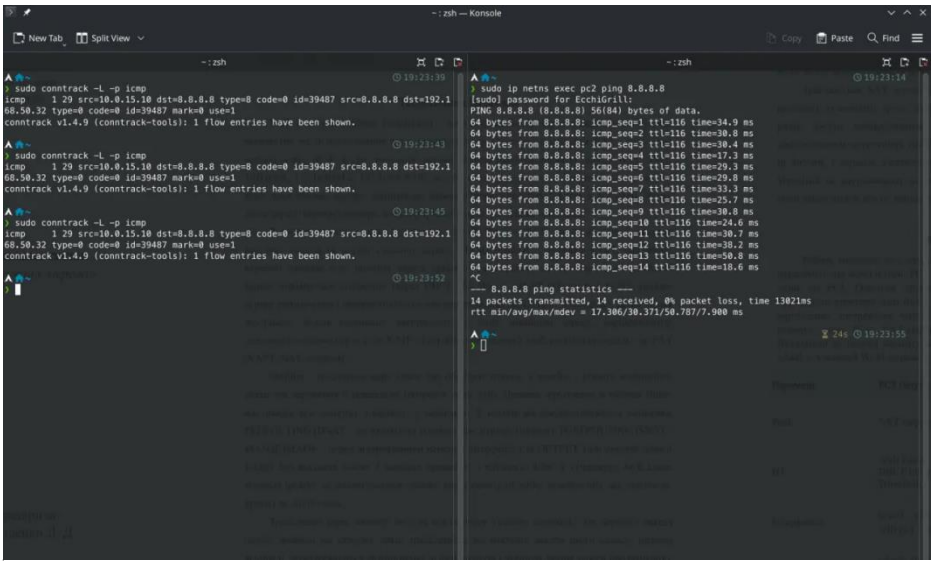


Рис. 12. Запис трансляції NAT у таблиці conntrack.

10.Перевірка локального зв’язку між PC1 та PC2

Очищаю ARP-кеш, запускаю захоплення у Wireshark на veth-pc1 з фільтром arp || icmp і виконую ping в обидва боки.

```
sudo ip neigh flush dev veth-pc1
ping -c 4 10.0.15.10
sudo ip netns exec pc2 ping -c 4 10.0.15.1
```

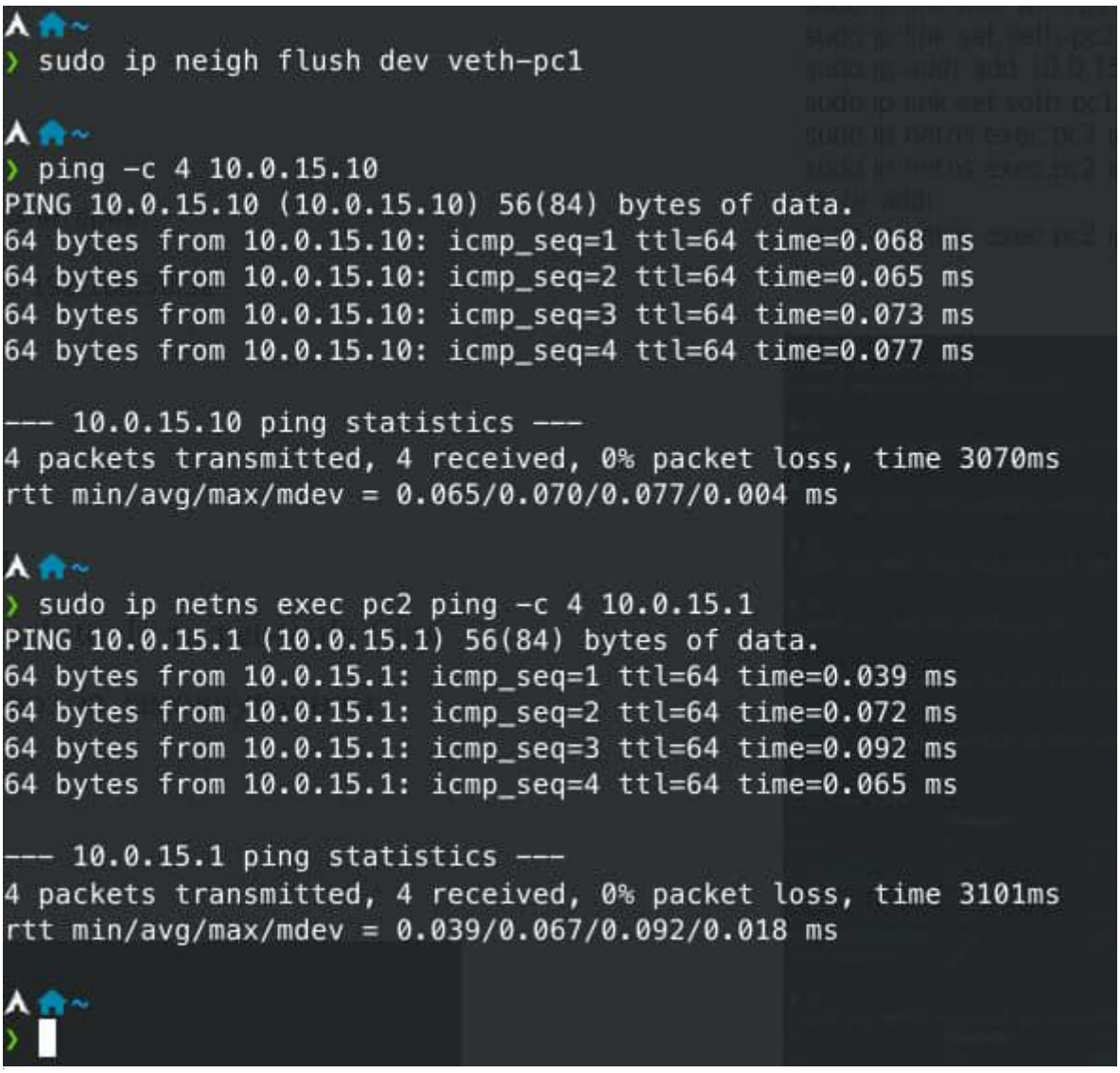


Рис. 13. Перевірка локального зв’язку PC1 <=> PC2

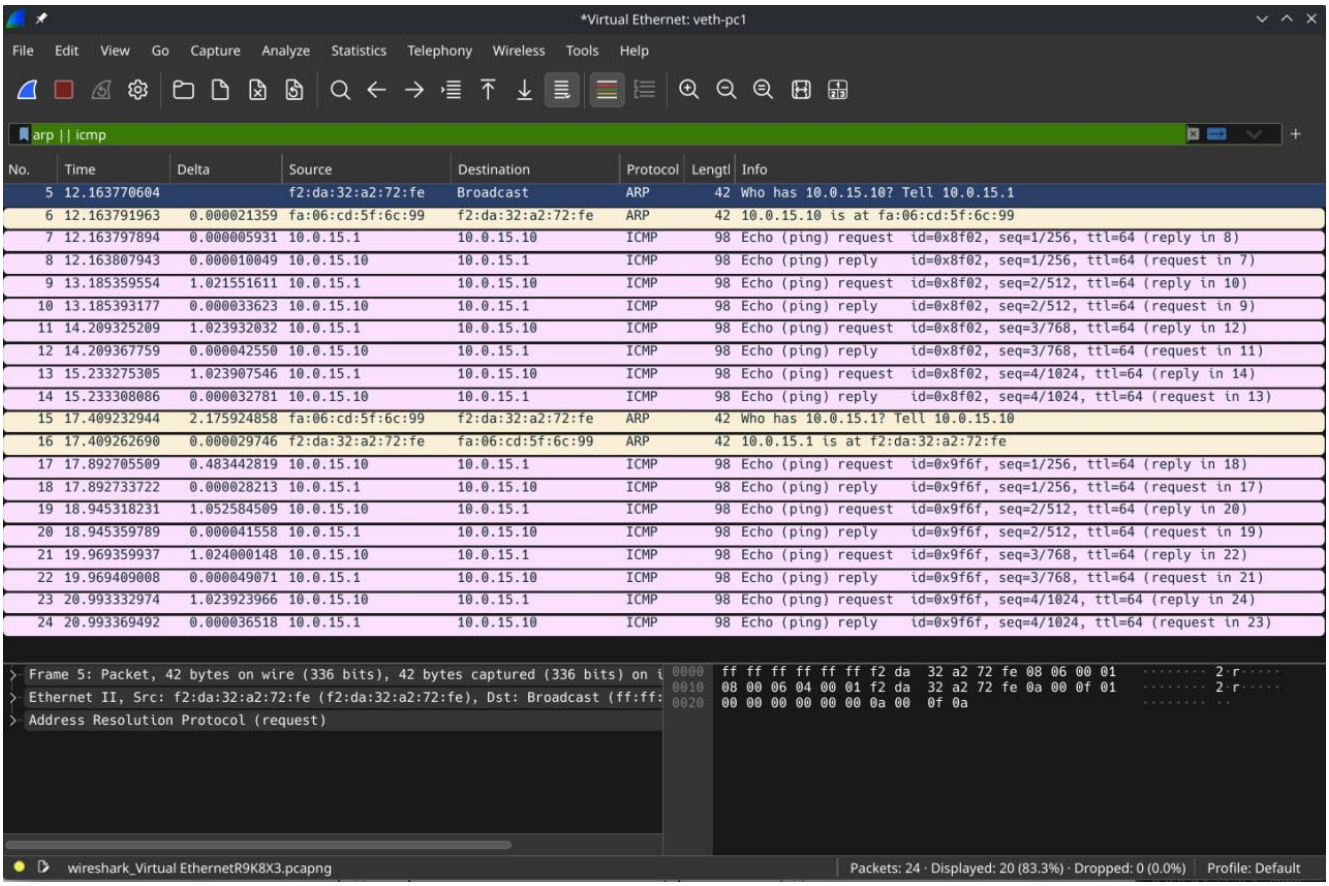


Рис. 14. ARP-запит і відповідь між PC1 та PC2

11.Перевірка доступу PC1 до Інтернету

```
ping -c 4 google.com
```

```
~
❯ ping -c 4 google.com
PING google.com (142.250.130.113) 56(84) bytes of data.
64 bytes from zq-in-f113.1e100.net (142.250.130.113): icmp_seq=1 ttl=114 time=16.5 ms
64 bytes from zq-in-f113.1e100.net (142.250.130.113): icmp_seq=2 ttl=114 time=16.6 ms
64 bytes from zq-in-f113.1e100.net (142.250.130.113): icmp_seq=3 ttl=114 time=22.4 ms
64 bytes from zq-in-f113.1e100.net (142.250.130.113): icmp_seq=4 ttl=114 time=41.7 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 16.475/24.287/41.720/10.341 ms

~
❯
```

Рис. 15. Доступ PC1 до Інтернету

12.Перевірка таблиць маршрутизації

```
route -n
sudo ip netns exec pc2 route -n
```

Для PC1 шлюзом за замовчуванням (Destination 0.0.0.0) є домашній роутер 192.168.50.1 через wlan0, а для мережі 10.0.15.0 шлюз не потрібен (0.0.0.0, інтерфейс veth-pc1), оскільки вона підключена безпосередньо. Для PC2 шлюзом за замовчуванням є PC1 - 10.0.15.1; цей шлюз PC2 отримав від DHCP-сервера.

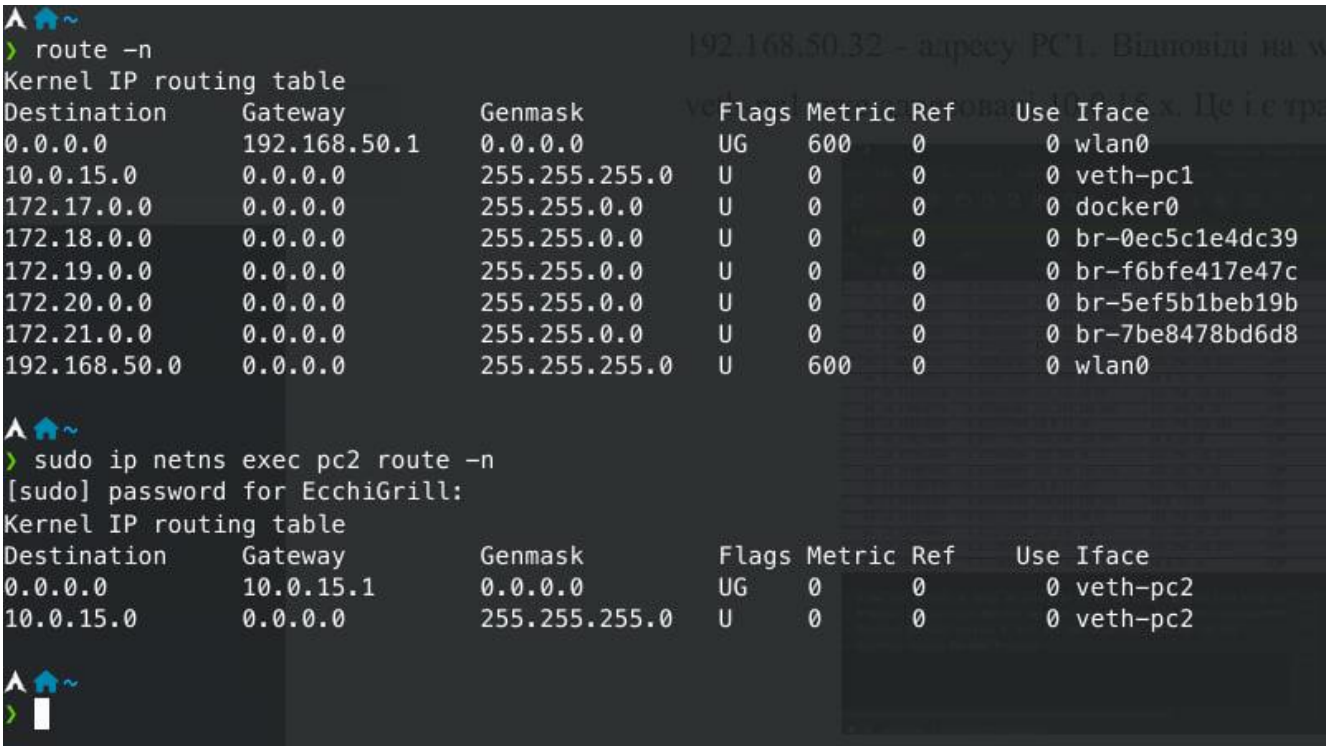


Рис. 16. Таблиці маршрутизації PC1 та PC2

13.Перевірка конфігурації DHCP-сервера та DNS

В Arch Linux конфігурація dhcpd знаходиться у /etc/dhcpd.conf. Перевіряю, що subnet відповідає мережі veth-pc1, routers = 10.0.15.1, а адреса DNS-сервера збігається з визначеною у Wireshark (192.168.50.1).

```
cat /etc/dhcpd.conf
sudo ip netns exec pc2 cat /etc/resolv.conf
```

```
~
❯ cat /etc/dhcpd.conf
default-lease-time 600;
max-lease-time 7200;
option subnet-mask 255.255.255.0;
option broadcast-address 10.0.15.255;
option routers 10.0.15.1;
option domain-name-servers 192.168.50.1, 8.8.8.8;
option domain-name "mydomain.example";

subnet 10.0.15.0 netmask 255.255.255.0 {
    range 10.0.15.10 10.0.15.20;
}

~
❯ sudo ip netns exec pc2 cat /etc/resolv.conf
nameserver 192.168.50.1
```

Рис. 17. Перевірка параметрів DHCP та DNS

14.Перевірка стану DHCP-сервера

В Arch Linux служба називається dhcpd4. Якщо служба неактивна, її перезапускаю командою sudo systemctl restart dhcpd4 або запускаю сервер вручну: sudo dhcpd -4 -cf /etc/dhcpd.conf veth-pc1.

```
systemctl status dhcpd4 --no-pager
sudo journalctl -u dhcpd4 -n 15 --no-pager
```

```
~
❯ systemctl status dhcpd4 --no-pager
● dhcpd4.service - IPv4 DHCP server
     Loaded: loaded (/usr/lib/systemd/system/dhcpd4.service; disabled; preset: disabled)
     Active: active (running) since Tue 2026-11-03 18:55:30 EET; 44min ago
 Invocation: 5aa290217edf44a79ffedc25cfb0d4be
   Main PID: 34317 (dhcpd)
      Tasks: 1 (limit: 37536)
     Memory: 4.6M (peak: 5.3M)
        CPU: 38ms
     CGroup: /system.slice/dhcpd4.service
             └─34317 /usr/bin/dhcpd -4 -q -cf /etc/dhcpd.conf -pf /run/dhcpd4/dhcpd.pid

Nov 03 19:21:00 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:21:00 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:25:23 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:25:23 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:29:40 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:29:40 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:34:34 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:34:34 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:38:40 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:38:40 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1

~
❯ sudo journalctl -u dhcpd4 -n 15 --no-pager
Nov 03 19:07:44 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:11:54 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:11:54 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:16:14 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:16:14 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:21:00 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:21:00 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:25:23 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:25:23 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:29:40 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:29:40 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:34:34 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:34:34 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:38:40 EcchiGrill dhcpd[34317]: DHCPREQUEST for 10.0.15.10 from fa:06:cd:5f:6c:99 via veth-pc1
Nov 03 19:38:40 EcchiGrill dhcpd[34317]: DHCPACK on 10.0.15.10 to fa:06:cd:5f:6c:99 via veth-pc1
```

Рис. 18. Стан та журнал DHCP-сервера

15.Перевірка правил NAT та маршрутизації на PC1

Лічильники pkts / bytes правила MASQUERADE зростають після трафіку з PC2 - отже, правило спрацьовує. Також перевіряю, що ip_forward увімкнено.

```
sudo iptables -t nat -L POSTROUTING -n -v
sysctl net.ipv4.ip_forward
```

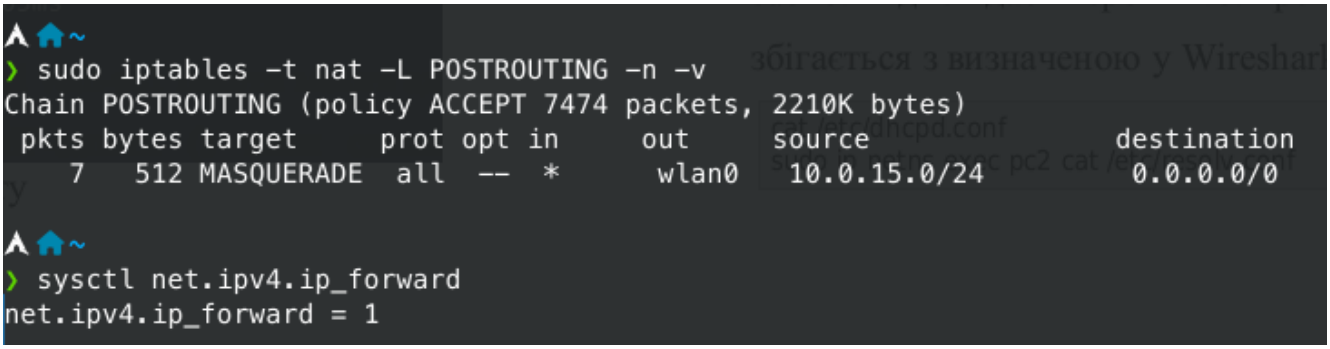


Рис. 19. Перевірка правил iptables NAT і параметра ip_forward

16.Перевірка зв’язку PC2 з DNS-сервером

Виконую ping DNS-сервера з PC2 і паралельно переглядаю у Wireshark на veth-pc1 стовпці Source та Destination, а також DNS-запити клієнта (фільтр icmp || dns).

```
sudo ip netns exec pc2 ping -c 4 192.168.50.1
sudo ip netns exec pc2 ping -c 4 8.8.8.8
```

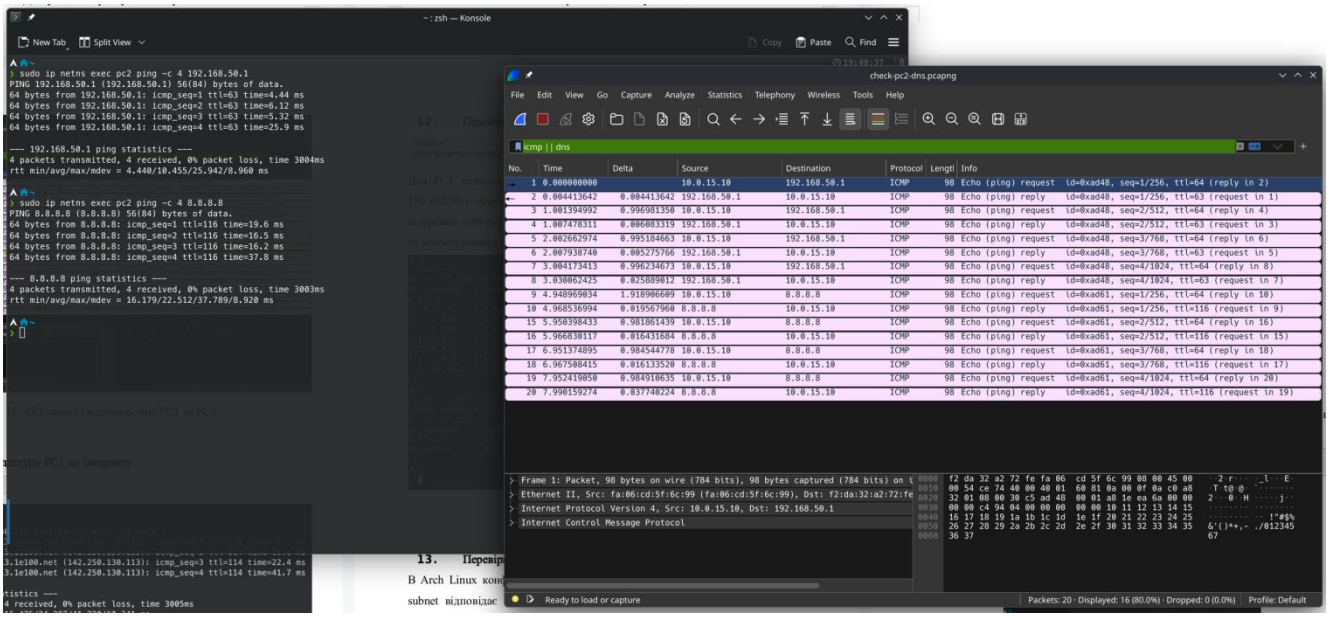


Рис. 20. Перевірка зв’язку PC2 з DNS-серверами

Відтворення дефекту та defect-report

Під час виконання збереження параметра маршрутизації на Arch Linux виявлено дефект: рекомендований рядок у /etc/sysctl.conf не застосовується після перезавантаження, оскільки systemd-sysctl читає лише каталоги sysctl.d. Перезавантаження моделюю скиданням параметра та повторним запуском служби systemd-sysctl, яка під час завантаження системи застосовує збережені параметри.

```
sudo mv /etc/sysctl.d/30-ipforward.conf /root/
echo "net.ipv4.conf.all.forwarding=1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -w net.ipv4.ip_forward=0
sudo systemctl restart systemd-sysctl
sysctl net.ipv4.ip_forward
sudo ip netns exec pc2 ping -c 4 10.0.15.1
sudo ip netns exec pc2 ping -c 4 8.8.8.8
```

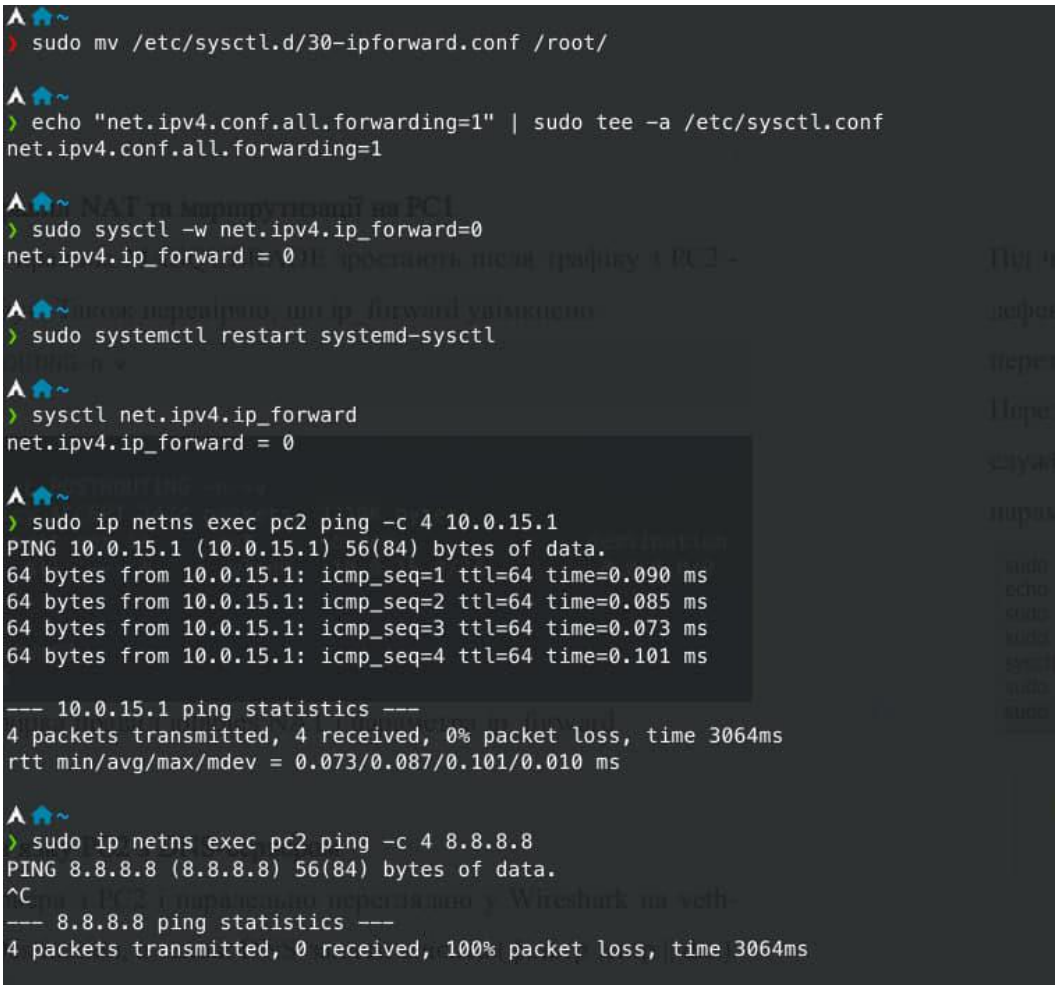


Рис. 21. Відтворення дефекту: маршрутизація вимкнена після «перезавантаження»

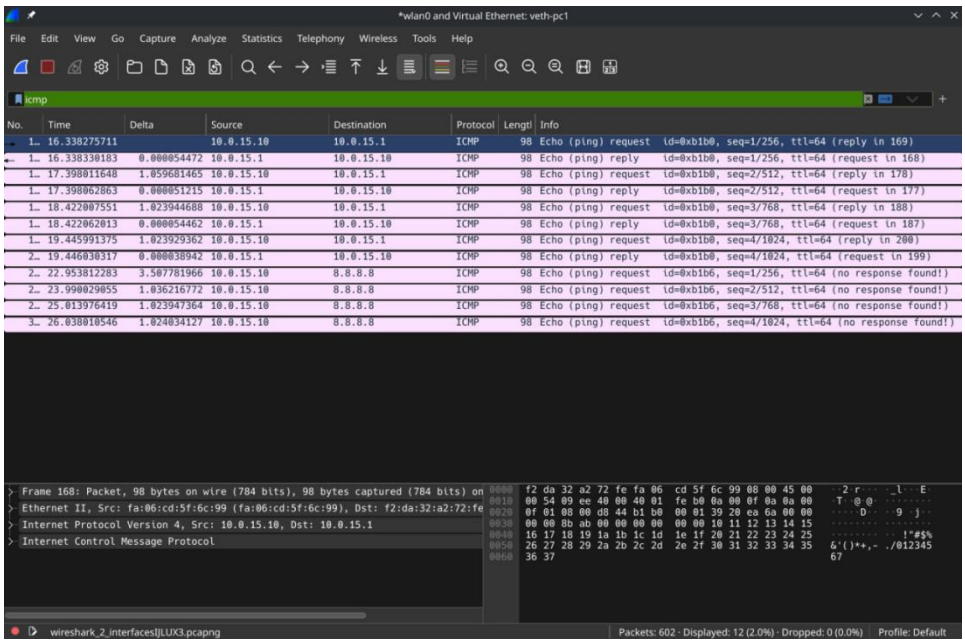
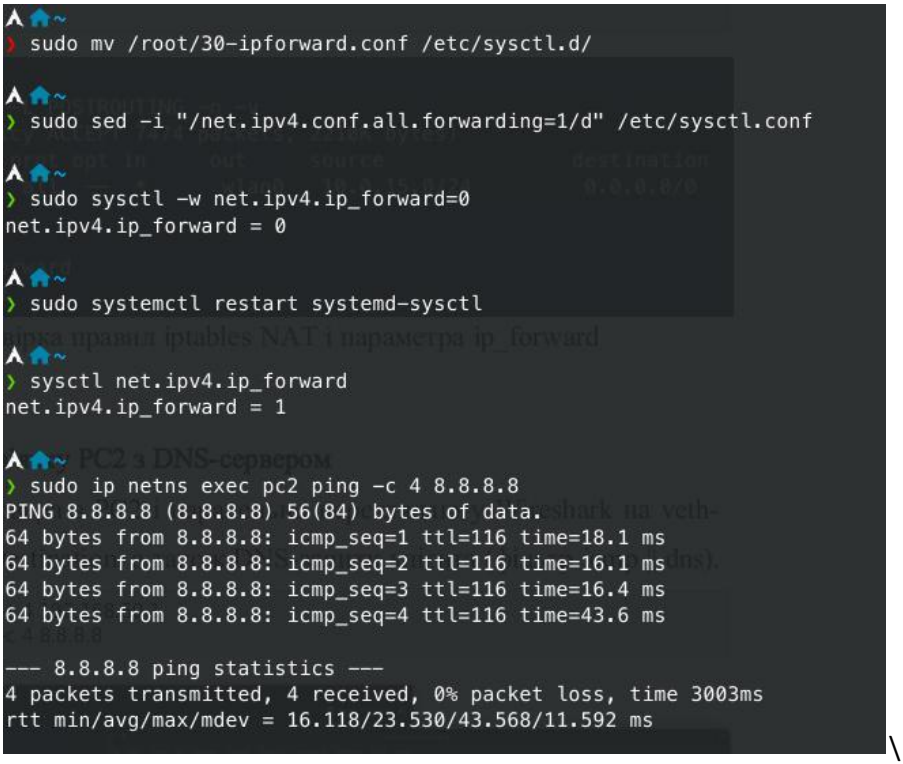


Рис. 22. Локалізація дефекту у Wireshark

Виправлення та повторна перевірка:

```
sudo mv /root/30-ipforward.conf /etc/sysctl.d/
sudo sed -i "/net.ipv4.conf.all.forwarding=1/d" /etc/sysctl.conf
sudo sysctl -w net.ipv4.ip_forward=0
sudo systemctl restart systemd-sysctl
sysctl net.ipv4.ip_forward
sudo ip netns exec pc2 ping -c 4 8.8.8.8
```



\

Рис. 23. Перевірка виправлення дефекту

Defect-report

Поле	Значення
ID	BUG-NAT-001
Назва	Після перезавантаження PC1 клієнт внутрішньої мережі втрачає доступ до Інтернету: параметр маршрутизації з /etc/sysctl.conf не застосовується в Arch Linux
Компонент	Налаштування NAT-маршрутизатора / параметри ядра (sysctl)
Середовище	PC1: ноутбук, Arch Linux, systemd-sysctl, iptables (nf_tables), інтерфейси wlan0 (192.168.50.32/24) та veth-pc1 (10.0.15.1/24) PC2: network namespace pc2, veth-pc2 (10.0.15.x/24, шлюз 10.0.15.1)
Серйозність (Severity)	Major
Пріоритет (Priority)	High
Передумови	1. Налаштовано NAT згідно з інструкцією: правило MASQUERADE для 10.0.15.0/24 через wlan0. 2. Параметр маршрутизації збережено лише рядком net.ipv4.conf.all.forwarding=1 у /etc/sysctl.conf (як у п. 3 інструкції), файлу в /etc/sysctl.d/ немає. 3. До перезавантаження PC2 має доступ до Інтернету.
Кроки відтворення	1. Перезавантажити PC1 (або змоделювати: sudo sysctl -w net.ipv4.ip_forward=0; sudo systemctl restart systemd-sysctl). 2. Виконати sysctl net.ipv4.ip_forward на PC1. 3. З PC2 виконати ping 10.0.15.1 та ping 8.8.8.8. 4. Переглянути ICMP-трафік у Wireshark на veth-pc1 і wlan0.
Очікуваний результат	Після перезавантаження net.ipv4.ip_forward = 1, PC1 пересилає та транслює пакети, ping 8.8.8.8 з PC2 успішний.
Фактичний результат	net.ipv4.ip_forward = 0. Ping шлюзу 10.0.15.1 з PC2 успішний, а ping 8.8.8.8 - 100% packet loss. У Wireshark ICMP-запити PC2 є на veth-pc1, але відсутні на wlan0: PC1 відкидає пакети, не адресовані йому, і правило MASQUERADE до них не застосовується. Лічильники правила MASQUERADE не зростають.
Наслідки	Після кожного перезавантаження маршрутизатора всі вузли внутрішньої мережі втрачають доступ до Інтернету, хоча правила iptables відновлено і локальна мережа працює; проблема неочевидна, бо ping шлюзу проходить успішно.
Причина	Служба systemd-sysctl в Arch Linux зчитує параметри лише з каталогів /etc/sysctl.d/, /run/sysctl.d/, /usr/lib/sysctl.d/; файл /etc/sysctl.conf не обробляється. Інструкція орієнтована на Debian/Ubuntu (так само як і рядок post-up у /etc/network/interfaces для відновлення правил iptables).

Поле	Значення
Рекомендації	Зберігати параметр у файлі /etc/sysctl.d/30-ipforward.conf (net.ipv4.ip_forward = 1) і застосовувати командою sudo sysctl --system; правила iptables зберігати в /etc/iptables/iptables.rules з увімкненою службою iptables.service.

Висновок: У роботі досліджено технологію NAT і налаштовано NAT-маршрутизатор на ноутбуці з Arch Linux за допомогою iptables. Другий комп’ютер змодельовано мережевим простором імен, з’єднаним з основною системою парою veth, а адресацію внутрішньої мережі 10.0.15.0/24 (варіант 15) забезпечено DHCP-сервером ISC dhcpd. Після увімкнення маршрутизації в ядрі (net.ipv4.ip_forward) і додавання правила MASQUERADE у ланцюжок POSTROUTING клієнт PC2 отримав доступ до Інтернету. За допомогою Wireshark на внутрішньому та зовнішньому інтерфейсах підтверджено заміну адреси джерела 10.0.15.x на адресу PC1, а таблиця conntrack показала відповідний запис трансляції. Траблшутинг виконано за рівнями: локальний зв’язок і ARP, доступ PC1 до Інтернету, таблиці маршрутизації, параметри DHCP і DNS, стан служби, правила NAT та зв’язок клієнта з DNS. Під час перевірки збереження налаштувань виявлено дефект: в Arch Linux параметр із /etc/sysctl.conf не застосовується після перезавантаження, через що NAT перестає працювати; дефект оформлено у вигляді defect-report і виправлено перенесенням параметра в /etc/sysctl.d/. Отже, для стабільної роботи NAT потрібні три складові - увімкнена маршрутизація, правило трансляції та коректне збереження обох налаштувань з урахуванням особливостей дистрибутива.